Live Network Fault Detection

Complete project instruction manual

A practical record of the lab, telemetry, fault experiments, model training, live scoring, evaluation, troubleshooting, and lessons learned.

Project scope
A local Docker and Containerlab prototype that measures a routed network, injects controlled faults, classifies telemetry, and evaluates predictions against recorded fault windows.

How to read this guide
Commands are shown in execution order. Each step explains what it does, why it is needed, and when to run it. Results are tied to their test conditions; they are not production guarantees.

Prepared from the project files, saved evaluation artifacts, and our project discussion
October 2026

Guide contents

- The project journey and decisions
- Questions we clarified during the project
- How to use this manual
- 1. What the project does
- 2. Project route map
- 3. How a live prediction is made
- 4. Tools and what each one does
- 5. Important files and directories
- 6. Prepare and start the lab
- 7. Collect data and run the live detector
- 8. Evaluate, train, and interpret
- 9. Problems encountered and resolutions
- 10. Results reached
- 11. Development roadmap
- 12. Common command reference
- 13. What remains and limitations

This guide explains the project from the network lab through live predictions, evaluation, and lessons learned. It is written for someone who wants to understand, run, explain, and extend the project, not only copy commands.

1. What the project does

The project creates a small routed network, measures its behavior, deliberately injects network faults, and uses an XGBoost classifier to identify fault types from telemetry.

The fault labels are **not given to the live detector**. The detector reads measurements and predicts a class. A separate fault schedule records when faults were injected so that the prediction can be scored afterward.

The result is a live lab prototype for detecting:

- Normal operation
- High latency
- Packet loss
- Congestion
- Link down

It is a demonstration and evaluation project, not a production network monitoring service.

The project journey and decisions

This section records the sequence of engineering decisions reflected in the project files, evaluation artifacts, and our project discussion. It is a reconstruction of the work, not a verbatim chat transcript. Exact timestamps are provided only where the saved artifacts support them.

Stage A - Define the goal and understand the signal

The goal was to detect and classify network faults in a small lab. We clarified a key distinction: at first, the classifier was being scored against saved data; after the streaming detector was added, it could also emit predictions while the lab was running. A live prediction is not the same thing as a live production deployment. In this project, "live" means telemetry is sampled and scored during an active Containerlab experiment on the local Docker engine.

We also clarified how a fault is identified. The detector does not read the injected line or the fault schedule to make ordinary class predictions. It looks at measured changes such as counters, RTT, ping loss, link state, and throughput. The schedule is used after the run to identify the expected label and calculate evaluation metrics. Link-down is a deliberate exception: an observed kernel interface state of DOWN is strong direct evidence and triggers an explicit detector rule.

Stage B - Make a repeatable network lab

A six-node diamond topology was chosen: two hosts and four FRR routers, with two paths between the endpoints. The redundant route makes the lab realistic enough to demonstrate failover, but it also creates an experimental trap: if a fault is injected on a path no longer carrying traffic, the configured fault and the observed behavior do not match. That lesson shaped the later test order and label-validity checks.

Stage C - Collect measured telemetry and inject controlled faults

The lab was extended to collect timestamped interface counters and state, short RTT/loss bursts, and throughput while a background iperf3 flow generated traffic. The fault script applied link-down, latency, loss, and rate-limit conditions and wrote start/end intervals. Separate per-run directories were used so data, labels, logs, profile settings, and live predictions could be traced to one experiment.

Stage D - Replace a failed synthetic-to-real assumption

An early synthetic-to-real experiment scored only 0.203 macro F1, had a 45.2% normal false-alarm rate, and found no high-latency or packet-loss samples (zero recall for those classes). This was evidence that synthetic patterns did not represent the lab telemetry closely enough. The project then trained its main model from measured Containerlab captures and preserved the failed transfer as a limitation instead of hiding it.

Stage E - Fix evaluation leakage

An ordinary random row split can put highly similar samples from one capture into both training and test sets. That measures recognition of a familiar run more than generalization to a new experiment. Training was changed to split by complete run using GroupShuffleSplit, with 20% of eligible runs reserved for testing. A locked holdout run is marked training_eligible=false before any retraining, so its evaluation remains independent.

Stage F - Correct invalid fault labels caused by route failover

A link-down experiment on the r2-r4 path could move traffic onto r3-r4. Any later latency or loss injection on a bypassed path was not necessarily affecting the measured flow even if a script scheduled it. The solution was to disable r2-r4 for the clean profile so traffic uses r3-r4 for its later tests, inspect the injected link and qdisc, and exclude invalid capture segments from all-class claims. This is why label integrity matters as much as model code.

Stage G - Add live scoring and strengthen the data path

Live mode was integrated into the collection run. The collectors and detector operate alongside traffic and injections; timestamped prediction, confidence, alert state, and scoring time are saved. Telemetry features include rates and ratios derived from kernel counters, and auxiliary samples are aligned to the nearest measurement within eight seconds. Missing-value indicators keep absent samples visible to the model.

A counter-source problem was resolved by reading Linux kernel counters with `ip -s link` inside router containers rather than relying on incomplete router CLI statistics. An iperf3 image-build change made throughput generation available in the isolated lab hosts. The collection script gained preflight checks and cleanup so temporary link and traffic-shaping changes are undone after a run.

Stage H - Test beyond familiar settings and compare versions

The project tested a 500 ms / 18% loss / 1 Mbit/s profile, a link-down event at an unused r1-r3 location, and two repeated 450 ms / 15% loss / 1.5 Mbit/s profiles. These tests answer different questions: a grouped test estimates new-run performance among eligible profiles; a locked profile probes an unseen combination; a held-out location probes one fault class at a new topology location. They should not be collapsed into one universal accuracy claim.

On the first 450 ms capture, the newer model improved accuracy from 0.571 to 0.968 and macro F1 from 0.400 to 0.899 versus the older model on the same capture. This is a useful paired comparison, but the two repeated runs remain a small sample. Packet-loss estimates were especially noisy because each RTT collector sample used five pings.

Stage I - Make the project reproducible and explain its limits

Pinned dependencies, per-run artifacts, evaluation reports, setup steps, and security exclusions were prepared. Containerlab runtime state included a generated private certificate authority key, so generated runtime directories were excluded from the shareable source snapshot. Machine-specific copied scripts and the local Python environment were also excluded. These cleanup actions protect secrets and make a fresh setup more understandable.

Questions we clarified during the project

Was this running on a live server?

The evidence describes a live local lab experiment: Docker containers run under the Docker engine connected to Ubuntu WSL; telemetry is collected and scored while the experiment runs. It is not evidence of deployment to a remote server, a production network, or an always-on monitoring service.

Does it identify a fault from the injected command?

For normal prediction, it uses measured data, not the command line that injected a fault. The command and schedule establish expected labels for later evaluation. The direct observed-interface-DOWN rule is based on measured state. This separation prevents the offline evaluator from being mistaken for the live detector.

Is the project finished?

The controlled lab prototype and its reported evaluations are complete as a portfolio project. The result is not production-ready AIOps. More independent runs, longer loss measurements, deployment hardening, monitoring/alert delivery, and tests on a real or independently generated network would be needed before making stronger operational claims.

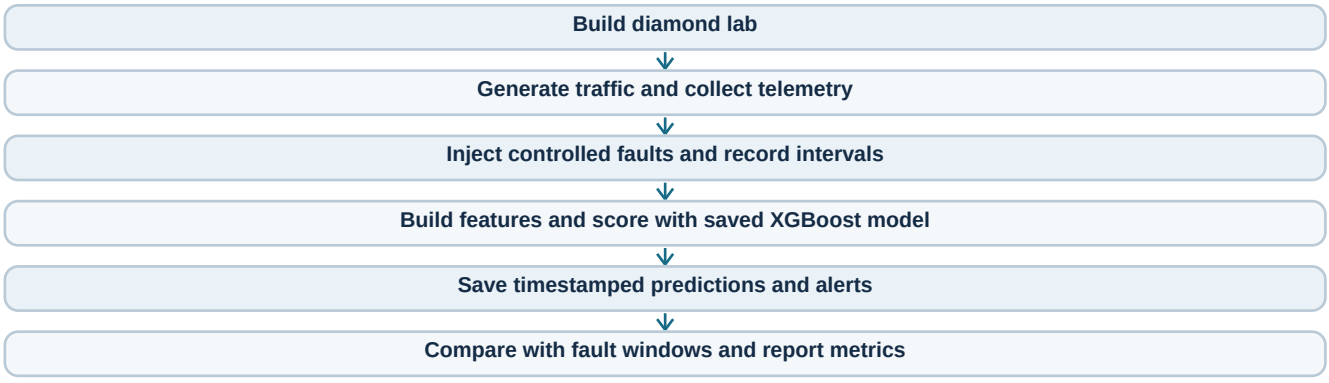
What does the accuracy mean?

The 0.991 accuracy belongs to the run-grouped test over 5,080 rows from seven held-out runs among the 35 eligible measured runs. It is not a guarantee for every unseen location or severity. The two repeated interpolation holdouts pooled to 0.972 offline accuracy / 0.919 macro F1 and 0.965 live accuracy / 0.905 macro F1. Per-class results, support, and false alarms reveal behavior hidden by overall accuracy.

How to use this manual

Follow Sections 6 and 7 when creating the environment and running the experiment. Use Section 8 when preserving an independent holdout, training, or scoring a capture. Use Sections 9 and 10 to understand why the experimental design changed and what the evidence currently supports. The command reference is a quick lookup; explanations next to each step describe what the command does, why it is needed, and when to run it. Do not skip the route, health, and label checks just because a command completed successfully.

2. Project route map



The detector uses telemetry plus the saved model. The fault schedule is used only to label and evaluate the results afterward.

Network topology

The lab is a diamond: traffic from `h1` to `h2` can pass through either `r2` or `r3`.

Topology route: h1 - r1 - (r2 or r3) - r4 - h2. The two middle branches create a diamond and allow path failover.

Link	Interfaces
h1-r1	`h1:eth1` ↔ `r1:eth1`
r1-r2	`r1:eth2` ↔ `r2:eth1`
r1-r3	`r1:eth3` ↔ `r3:eth1`
r2-r4	`r2:eth2` ↔ `r4:eth2`
r3-r4	`r3:eth2` ↔ `r4:eth3`
r4-h2	`r4:eth1` ↔ `h2:eth1`

3. How a live prediction is made

1. `collect_dataset.sh` verifies that all six lab containers are running and

that `h1` can reach `h2`.

2. It starts a long-running `iperf3` flow so the lab has traffic to measure.

3. Four collectors append timestamped rows every five seconds: interface

counters, interface state, RTT/loss, and throughput.

4. With `LIVE_DETECTOR=1`, `live_detector.py` polls the growing files every five

seconds and scores new telemetry rows with the saved model.

5. The detector writes `predicted`, `confidence`, `alert`, and scoring time to

`live_predictions.csv`, and prints predictions to `live_detector.log`.

6. The fault script records injection start/end times in `faults.csv`.

7. `evaluate_holdout_profile.py` later joins those time windows to telemetry

and compares predictions with the known injected faults.

Features include packet/byte/error/drop counter rates and ratios, RTT, measured ping loss, link state, throughput, and router/interface categories. Auxiliary streams are aligned to telemetry using the nearest sample within eight seconds. Missing auxiliary values receive explicit missing indicators.

There is one explicit domain rule: if the observed interface state is DOWN, the detector returns `link_down` with 100% confidence. This uses measured interface state, not `faults.csv` or the injection command.

4. Tools and what each one does

Tool	Use in this project
Docker Desktop + Docker CLI	Runs the Linux containers and lets scripts execute commands inside them.
WSL 2 / Ubuntu	Linux shell used for Containerlab, collection scripts, Python, and Git.

Tool	Use in this project
Containerlab	Builds the six-node topology from `diamond.clab.yml`.
FRRouting (FRR)	Provides routing on routers `r1-r4`; configs are in `r1-frr.conf` through `r4-frr.conf`.
Linux `ip` tools	Read interface counters/state and bring a test link down/up.
Linux `tc` (`netem`, `tbf`)	Adds delay/loss and limits link throughput for controlled fault injection.
`ping`	Measures end-to-end RTT and estimates packet loss using five-packet bursts.
`iperf3`	Generates sustained traffic and supplies throughput observations.
Python	Runs training, feature assembly, streaming detection, and evaluation.
pandas / NumPy	Reads, aligns, cleans, and transforms timestamped telemetry.
scikit-learn	Group splitting, class weighting, and evaluation metrics.
XGBoost	Trains and serves the multi-class fault classifier.

5. Important files and directories

Path	Purpose
`diamond.clab.yml`	Defines hosts, routers, links, and the diamond topology.
`Dockerfile.hosts`	Builds the lab host image with `iperf3`.
`r1-frr.conf` … `r4-frr.conf`	FRR routing configuration for each router.
`collect_dataset.sh`	Preflight checks, starts collectors/traffic/detector, runs a fault sequence, and saves a unique run directory.
`telemetry_collect_enhanced_v2.sh`	Reads packet, byte, error, and drop counters from the kernel using `ip -s link`.
`link_state_collect_v2.sh`	Records interface operational state and carrier.
`rtt_collect_v3.sh` / generated burst collector	Collects RTT and ping-loss samples.
`inject_faults_enhanced_v2.sh`	Injects link-down, latency, packet-loss, and congestion faults, and writes `faults.csv`.
`live_detector.py`	Builds features from current telemetry and emits live class predictions/alerts.
`train_model_xgboost_runs.py`	Trains XGBoost from eligible measured runs and holds out complete runs.
`evaluate_holdout_profile.py`	Evaluates a profile excluded from training; can evaluate only one fault class.
`requirements.txt`	Pinned Python library versions used by the current environment.
`GENERALIZATION_REPORT.md`	Full metrics, capture validity notes, and model limitations.
`dataset_runs/run_*`	Each separate capture: sensor CSVs, fault windows, and live predictions.
`model_runs/xgb_35run_20261013T082155Z/`	Current model, metrics, class list, and feature schema.
`model_runs/xgb_holdout_profile_*/`	Saved evaluation metrics and row-level predictions.

Root-level CSVs and older training scripts preserve earlier experiments. The current workflow uses per-run data under `dataset_runs/` and the `xgb_35run` model by default.

6. Step-by-step: prepare and start the lab

Run these commands in the **Ubuntu WSL terminal**, from the project folder. Keep Docker/containerlab commands in the same environment so they use the same Docker engine and filesystem.

Step 1: Check Docker connectivity

```
cd ~/aiops-project
docker context ls
docker ps
```

`docker context ls` shows which Docker daemon the CLI targets. `docker ps` shows running containers. An empty `docker ps` means no containers are running in that selected engine; it does not mean that fault detection has started.

For Docker Desktop on Windows, start Docker Desktop and enable the Ubuntu distribution under **Settings -> Resources -> WSL Integration**. Then retry `docker ps` inside Ubuntu. Avoid switching between the Windows named-pipe context and the WSL Unix-socket context mid-run. See the official Docker WSL guide.

Step 2: Install project Python dependencies

```
cd ~/aiops-project
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The virtual environment stays local and is excluded by `.gitignore`.

Step 3: Build the traffic-generator image

```
docker build -t aiops-multitool:latest -f Dockerfile.hosts .
```

`-f` selects the Dockerfile. `-t` gives the built image the name expected by the topology. `iperf3` is installed into the image at build time so the lab does not depend on downloading software after the test hosts are isolated.

Step 4: Deploy the topology

```
sudo containerlab deploy -t diamond.clab.yml
docker ps --format 'table {{.Names}}\t{{.Status}}'
```

`-t` selects the topology file. Deployment should create `clab-aiops-diamond-h1`, `h2`, and routers `r1`-`r4`.

Step 5: Check reachability and traffic tools

```
docker exec clab-aiops-diamond-h1 ping -c 3 10.0.2.10
docker exec clab-aiops-diamond-h1 iperf3 --version
docker exec clab-aiops-diamond-h2 iperf3 --version
```

The collection script repeats important preflight checks itself. If `iperf3` is missing, rebuild the host image and redeploy the topology.

7. Step-by-step: collect data and run the live detector

Standard live collection

```
cd ~/aiops-project
LIVE_DETECTOR=1 PYTHON=./.venv/bin/python bash collect_dataset.sh 1
```

`1` means one complete run. `LIVE_DETECTOR=1` starts streaming predictions. `PYTHON=...` selects the project virtual environment. `MODEL_DIR` defaults to `model_runs/xgb_35run_20261013T082155Z/`.

The default run uses a 60-second pre-fault baseline, four 60-second faults with 30-second gaps, and a 60-second post-fault baseline: about eight minutes. The detector and collectors run at the same time as the network traffic and fault injection.

Repeat the verified 500 ms profile

This order disables `r2-r4`, allowing the flow to use the `r3-r4` path for the later latency and loss faults:

```
MODEL_DIR=model_runs/xgb_35run_20261013T082155Z \
LINK_DOWN_NODE=r2 LINK_DOWN_INTERFACE=eth2 LINK_DOWN_LOCATION=r2-r4 \
LATENCY_MS=500 PACKET_LOSS_PCT=18 CONGESTION_RATE=1mbit \
LIVE_DETECTOR=1 PYTHON=./.venv/bin/python bash collect_dataset.sh 1
```

Repeat the two-run interpolation profile

These are the settings used for the two repeated, locked evaluations:

```
BASELINE_SECONDS=30 \
LATENCY_MS=450 PACKET_LOSS_PCT=15 CONGESTION_RATE=1500kbit \
LINK_DOWN_NODE=r2 LINK_DOWN_INTERFACE=eth2 LINK_DOWN_LOCATION=r2-r4 \
LIVE_DETECTOR=1 PYTHON=./.venv/bin/python bash collect_dataset.sh 1
```

The collection prints the new `dataset_runs/run_*` path. Preserve a run that will be evaluated as a holdout before retraining. Add these metadata rows to that run's `run_metadata.csv`:

```
printf 'training_eligible,false\nexclusion_reason,locked_holdout_do_not_train\n' \
  >> dataset_runs/<run_id>/run_metadata.csv
```

Replace `<run_id>` with the directory name printed by the collector. The trainer skips runs marked `training_eligible,false`.

What is written for every capture

- `telemetry.csv`: timestamped per-interface kernel counters.
- `rtt.csv`: five-ping RTT/loss measurements.
- `link_state.csv`: timestamped interface states.
- `throughput.csv`: throughput from the background `iperf3` stream.
- `faults.csv`: injected fault class, location, severity, and UTC interval.
- `live_predictions.csv`: streaming predictions, confidence, alert flag, and

scoring timestamp (when live mode is enabled).

- `run_metadata.csv`: profile values, times, and training eligibility.
- `live_detector.log`, collector logs, and fault log: troubleshooting output;

local runtime logs are excluded from Git.

8. Step-by-step: evaluate, train, and interpret

Evaluate a locked holdout

```
./.venv/bin/python evaluate_holdout_profile.py dataset_runs/<run_id>
```

This evaluates all classes. To evaluate only one class, such as a held-out link location where routing bypassed later faults:

```
./.venv/bin/python evaluate_holdout_profile.py \
  dataset_runs/<run_id> --fault-class link_down
```

The evaluator writes `holdout_predictions.csv` and `metrics.json` into a new `model_runs/xgb_holdout_profile_*` directory. It reads `faults.csv` for labels; the live detector does not.

Train a new model (only after preserving holdouts)

```
./.venv/bin/python train_model_xgboost_runs.py
```

The trainer loads eligible measured run data, excludes runs marked `training_eligible,false`, and groups rows by run. It uses a 20% run-level holdout with `random_state=42`, so rows from one run cannot appear in both training and test sets. It uses class-balanced sample weights and writes the model, predictions, feature names, and metrics to a newly named `model_runs/xgb_<N>run_<timestamp>/` directory.

Do not retrain on a locked evaluation run and then continue reporting its score as an independent test. Once used for training, it is no longer a holdout.

Read the metrics correctly

- **Accuracy**: fraction of all rows classified correctly. Many normal rows

can make this look high.

- **Balanced accuracy**: average recall across classes; better shows minority

fault performance.

- **Macro F1**: average of per-class F1 with equal weight for each class.
- **Precision**: among predictions for a class, the fraction that was correct.
- **Recall**: among actual rows of a class, the fraction detected.
- **Normal false-alarm rate**: normal rows incorrectly flagged as faults.
- **Support**: number of labeled rows in that class. Small support means more

run-to-run uncertainty.

For a portfolio, report accuracy alongside balanced accuracy, macro F1, per-class results, and false-alarm rate. Do not claim that one accuracy number means every fault is reliably detected.

9. Problems encountered and how they were addressed

Problem	Why it mattered	Resolution / lesson
Docker showed an empty `docker ps` at the beginning.	The CLI could be pointed at an engine with no lab containers running.	Check `docker context ls`, start Docker Desktop, enable Ubuntu WSL integration, then deploy the topology. Later checks showed all six lab containers running.
Windows Docker and WSL Docker contexts looked different.	Windows uses a named pipe while WSL commonly uses `/var/run/docker.sock`; changing context can point commands at a different engine.	Run the project consistently from Ubuntu WSL and verify the engine with `docker ps` before deployment.
`iperf3` was missing in lab hosts.	Throughput telemetry could not be collected.	Add `iperf3` to `Dockerfile.hosts`, build `aiops-multitool:latest`, and redeploy.
Router CLI did not provide useful veth packet counters.	Counters from `vtysh` were incomplete for this lab.	Read kernel counters via `ip -s link` inside each router container.

Problem	Why it mattered	Resolution / lesson
Synthetic training data transferred poorly to the measured lab.	It did not resemble the real distribution of network signals. The synthetic-to-real test reached only 0.203 macro F1, a 45.2% normal false-alarm rate, and zero recall for high latency and packet loss.	Use measured Containerlab data for the main model. Keep the failed transfer result documented as a limitation.
A high score from a random row split overstated generalization.	Samples from one collection can be nearly identical and leak run conditions across splits.	Split by complete run using `GroupShuffleSplit`, and test separately on held-out profiles.
Link-down changed routing and bypassed later injections.	Later faults injected on `r3-r4` were labeled as active even though traffic had failed over to another path.	For clean all-class tests, disable `r2-r4` so `r3-r4` remains active. Mark invalid captures `training_eligible,false` and do not use them for training or all-class claims.
Ping-loss measurements were noisy.	Each sample contains only five pings; observed loss can differ from the injected percentage.	Report the observed signal and sample count; do not imply ping estimates exactly equal configured loss. Use additional repeats for stronger estimates.
Initial high accuracy did not mean every class was strong.	The 500 ms/18%/1 Mbit/s profile had weak packet-loss and congestion scores despite 0.929 accuracy.	Report per-class precision/recall/F1, compare repeated captures, and call this a prototype.
Containerlab generated a private CA key inside runtime state.	That key must never be placed in a shareable project snapshot.	Add `/clab-aiops-diamond/` to `.gitignore`; it is ignored and not part of the shareable source snapshot.

10. Results reached

Main model split

The current model is `model_runs/xgb_35run_20261013T082155Z/`. It used 35 eligible measured runs: 28 train runs and seven held-out runs (5,080 test rows).

Metric	Result
Accuracy	0.991
Balanced accuracy	0.979
Macro F1	0.973
False-alarm rate on normal rows	0.6%

This split evaluates new runs from collected profiles. It does not prove performance at every unseen severity or location.

Untouched 500 ms profile

`dataset_runs/run_001_20261013T082217Z/` used 500 ms latency, 18% packet loss, 1 Mbit/s congestion, and link-down on `r2-r4`. Its all-class evaluation was 0.929 accuracy, 0.802 balanced accuracy, 0.771 macro F1, and 5.3% normal false-alarm rate. Per-class F1: congestion 0.646, high latency 0.720, link-down 1.000, normal 0.973, packet loss 0.516. Accuracy alone hides the weakness in loss and congestion on this single profile.

Repeated 450 ms profile

Two separate live collections used 450 ms latency, 15% packet loss, and 1.5 Mbit/s congestion. Pooled results:

Evaluation	Rows	Accuracy	Balanced accuracy	Macro F1
Offline evaluator	1,061	0.972	0.927	0.919
Live detector, timestamp-aligned to labels	1,060	0.965	0.908	0.905

Pooled offline class F1: congestion 0.859, high latency 0.909, link-down 0.970, normal 0.988, packet loss 0.870. Packet-loss F1 varied from 0.778 on the first run to 0.970 on the second, so more independent runs would improve confidence in the estimate.

Held-out link-down location

Moving link-down to the unused `r1-r3` segment gave 1.000 precision and 0.944 recall on the valid event (17 of 18 positive samples detected, no false link-down alarms among other rows). Later fault events in that capture were excluded because routing bypassed their injection path.

Same-capture baseline comparison

On the first 450 ms held-out capture, the previous model scored 0.571 accuracy, 0.513 balanced accuracy, 0.400 macro F1, and a 43.6% normal false-alarm rate. The current model reached 0.968 accuracy and 0.899 macro F1 on that capture: 39.7 percentage points higher accuracy, 49.9 points higher macro F1, and 42.0 points fewer normal false alarms.

Full per-class reports and individual run artifacts are in `GENERALIZATION_REPORT.md` and `model_runs/xgb_holdout_profile_*/`.

11. Development roadmap: what changed over time

1. **Initial classifier experiments:** earlier scripts (`train_model.py`,

`train_model_v2.py`, and enhanced variants) established labels and basic model training. Synthetic data was useful for a first experiment but did not transfer to measured network telemetry.

2. **Build a measurable lab:** the Containerlab diamond and FRR routers gave

controlled paths where faults could be injected and observed.

3. **Replace synthetic-only evidence with measured runs:** collectors began

saving counters, RTT/loss, link state, throughput, and the fault schedule into unique `dataset_runs/run_*` folders.

4. **Improve data validity and evaluation:** complete runs-not random rows-are

held out during training. Captures where routing bypassed a later fault are marked ineligible. The clean link-down setting preserves the path for later latency/loss tests.

5. **Add streaming detection:** the live detector was wired into collection so

it scores telemetry as it arrives while the same rows are saved for audit and offline scoring.

6. **Test beyond familiar conditions:** a 500 ms/18%/1 Mbit/s severity profile,

a link-down on the unseen `r1-r3` segment, and two repeats at 450 ms/15%/1.5 Mbit/s tested transfer beyond the training profiles.

7. **Document limits and make it shareable:** per-class metrics, noisy sensor

estimates, valid/invalid captures, setup commands, pinned dependencies, and the generated-key exclusion were added to the reports and repository prep.

12. Common command reference

Command	Meaning
`docker context ls`	List Docker engines/contexts and the currently selected one.
`docker ps`	List running containers in the selected engine.
`docker exec <container> <command>`	Run a command inside a running container.
`docker build -t aiops-multitool:latest -f Dockerfile.hosts .`	Build the lab host image with `iperf3`.
`sudo containerlab deploy -t diamond.clab.yml`	Create the six-node lab.
`python3 -m venv .venv`	Create a project-local Python environment.
`source .venv/bin/activate`	Activate that environment in the current shell.
`python -m pip install -r requirements.txt`	Install pinned Python dependencies.
`LIVE_DETECTOR=1 ... bash collect_dataset.sh 1`	Collect one run and start streaming detection.
`./.venv/bin/python evaluate_holdout_profile.py <run_dir>`	Score one held-out capture against injection labels.
`./.venv/bin/python train_model_xgboost_runs.py`	Train a new run-grouped model from eligible captures.
`docker exec clab-aiops-diamond-r2 tc qdisc show dev eth2`	Inspect whether shaping is present on the r2 link.
`sudo containerlab destroy -t diamond.clab.yml`	Stop and remove the topology when finished.

The collection script installs an exit cleanup trap: it stops collectors and iperf3, brings the configured link back up, and deletes test qdiscs from `r2` and `r3`.

13. What remains and limitations

- The project is complete as a live lab prototype. It is not production AIOps.
- The 450 ms profile has two captures; the 500 ms profile and held-out link

location each have limited independent runs.

- Ping loss uses short five-packet samples and is noisy.
- Traffic routing can make an injected fault ineffective if it is applied to a

path that failover has bypassed; validate signal and route before trusting labels.

- The model has known per-class variation. Always include class metrics and

false-alarm rate with accuracy.

Detailed setup and code walkthrough

This expanded appendix explains the Windows, WSL, Docker Desktop, Ubuntu, Containerlab, Python, and Linux commands used by this project. It walks through the actual Dockerfile and topology structure, and explains the main safeguards in the data collector. GitHub publishing instructions are not included.

Scope: each command and important configuration line shown here is explained. Large third-party library internals and every line of every model-training script are summarized by their role; the source files remain the authoritative implementation.

Appendix contents

• Terminal basics

• WSL 2 and Ubuntu

• Docker Desktop integration

• Ubuntu tools and Containerlab

• Python environment

• Dockerfile and image build

• Containerlab topology

• FRR routing config

• Deploy and verify

• Linux telemetry commands

• Fault injection and cleanup

• Collector and live detector

• Troubleshooting and official references

1. Terminal basics

Run Windows management commands in Windows PowerShell. Run project and Linux commands in Ubuntu Bash (your Ubuntu WSL terminal). The project path in Ubuntu is /home/shihadlinux/aiops-project. Type only the command shown, not the prompt characters such as PS C:\> or user@host:~$.

```
cd ~/aiops-project
pwd
ls
```

Line or command	What it means and when to use it
cd ~/aiops-project	Change the current folder to the project under the Ubuntu home directory.
pwd	Print the current directory. Confirm it ends with /aiops-project before running project scripts.
ls	List files and folders; look for diamond.clab.yml, Dockerfile.hosts, requirements.txt, and collect_dataset.sh.

In Bash, ~ is shorthand for the Ubuntu home directory. A command prompt beginning with (aiops-env) or (.venv) indicates an active Python virtual environment.

2. WSL 2 and Ubuntu

The project history shows work in Ubuntu WSL. If Ubuntu is already installed, check it rather than reinstalling; reinstalling or unregistering a distribution can remove its local files.

```
wsl --status
wsl --list --verbose
wsl --update
```

Line or command	What it means and when to use it
wsl --status	Windows PowerShell command. Show installed WSL and default version information.
wsl --list --verbose	List distributions and their WSL version. Ubuntu should show version 2.
wsl --update	Update WSL and its Linux kernel. Restart Windows if requested.

If Ubuntu is not installed, open PowerShell as Administrator and run the following. First launch asks you to create a Linux username and password; this password is for Ubuntu sudo, not your Windows or Docker password.

```
wsl --install -d Ubuntu
```

Line or command	What it means and when to use it
wsl	Start the Windows Subsystem for Linux management utility.
--install	Install WSL components and a distribution.
-d Ubuntu	Choose Ubuntu. A restart may be required.

If an existing distribution shows version 1, convert it. Substitute its exact listed name if it is not Ubuntu:

```
wsl --set-version Ubuntu 2
```

Line or command	What it means and when to use it
wsl --set-version Ubuntu	Select the installed distribution to convert.
2	Set it to WSL 2, whose Linux kernel supports this Docker Desktop setup.

3. Docker Desktop integration with WSL

Docker Desktop runs the Docker engine on Windows. WSL integration exposes the Docker CLI and engine connection inside Ubuntu, so you can keep source code and Linux tools together. Docker warns against running a separate Docker Engine installed inside the same WSL distribution alongside Desktop integration because the two can conflict.

Windows GUI steps: install and start Docker Desktop; under Settings > General enable the WSL 2 based engine if shown; under Settings > Resources > WSL Integration enable Ubuntu and Apply/restart. Names may vary slightly by Desktop version.

```
docker context ls
docker version
docker ps
```

Line or command	What it means and when to use it
docker context ls	List Docker endpoints. The asterisk marks the selected context. In the project chat, default was unix:///var/run/docker.sock and desktop-linux showed a Windows named-pipe endpoint; run project commands consistently from Ubuntu.
docker version	Confirm both Docker client and server are reachable. A client-only display or socket error means Desktop is stopped or WSL integration is not working.
docker ps	List running containers. Empty output means Docker answered but no containers are running yet; deploy the topology next.

If Docker cannot connect: start Docker Desktop in Windows, enable Ubuntu in WSL Integration, close and reopen Ubuntu, then retry docker version. Do not change contexts at random: a context selects which Docker engine receives commands.

4. Ubuntu tools and Containerlab

apt manages Ubuntu system packages; pip manages Python libraries inside the virtual environment. Use apt for shell/network utilities and use pip for requirements.txt.

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip curl iproute2 iputils-ping
```

Line or command	What it means and when to use it
sudo	Request administrator privileges for the following Linux package command.
apt update	Refresh package lists; it does not itself install upgrades.
apt install -y	Install listed packages and automatically answer yes.
python3 python3-venv python3-pip	Python interpreter, virtual-environment support, and pip.
curl	Fetch installer resources over HTTPS.
iproute2	Supplies ip and tc, used for interface inspection and fault shaping.
iputils-ping	Supplies ping for reachability and RTT/loss samples.

Check whether Containerlab is already installed:

```
containerlab version
```

If missing, use the current official installation instructions. The documented convenience installer downloads and runs the official script; it installs system software and uses elevated privileges, so follow the current official page:

```
bash -c "$(curl -sL https://get.containerlab.dev)"
```

Line or command	What it means and when to use it
bash -c	Tell Bash to execute a string as a command.
$(...)	Run the enclosed curl command and substitute its output.
curl -sL URL	Download quietly and follow redirects.
https://get.containerlab.dev	Official Containerlab installer endpoint.

Containerlab creates container nodes and virtual links from the topology YAML. It needs Linux privileges and access to a functioning Docker engine.

5. Project Python environment

The checked-in requirements.txt pins NumPy 2.5.3, pandas 3.0.6, scikit-learn 1.9.1, and XGBoost 3.4.1. The recorded environment used Python 3.14.4. On a clean machine, use a Python version supported by all pinned wheels; if pip cannot find a compatible wheel, install a supported Python version and recreate .venv.

```
cd ~/aiops-project
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip check
```

Line or command	What it means and when to use it
cd ~/aiops-project	Move to project root so .venv and relative paths are created in the right place.
python3 -m venv .venv	Run Python module venv to create an isolated environment named .venv.
source .venv/bin/activate	Activate that environment in this Bash session; python and pip resolve to its copies.
python -m pip install --upgrade pip	Update pip inside .venv.
python -m pip install -r requirements.txt	Install each pinned library version from the requirements file.
python -m pip check	Check that installed packages have compatible dependencies.

Use deactivate to leave the environment. If you open a new terminal, activate again. Alternatively the collection command sets PYTHON=./.venv/bin/python explicitly.

6. Dockerfile and image build

Dockerfile.hosts is used to make the image for h1 and h2. It installs iperf3 while the image is being built, before the hosts enter the lab network.

```
FROM praqma/network-multitool:latest

# iperf3 is not reliably present in this image, so install it at build time.
RUN apk add --no-cache iperf3
```

Line or command	What it means and when to use it
FROM praqma/network-multitool:latest	Choose the base image and inherit its filesystem and tools. latest can change upstream, so future rebuilds may differ.
# ...	Comment only; Docker does not execute it. It records why iperf3 is added.
RUN apk add --no-cache iperf3	Execute a command during image build. apk is Alpine Linux package management; add installs iperf3; --no-cache avoids retaining package-index cache.

```
docker build -t aiops-multitool:latest -f Dockerfile.hosts .
```

Line or command	What it means and when to use it
docker build	Ask Docker to build an image.
-t aiops-multitool:latest	Tag it with the name and tag referenced by diamond.clab.yml.
-f Dockerfile.hosts	Select the project Dockerfile instead of the default Dockerfile.
.	Use current directory as the Docker build context.

7. Containerlab topology line by line

The YAML file defines six Linux containers and their links. Indentation shows which keys belong to which node. The following is a shortened, valid excerpt for explanation, not a complete deployable copy; the real file has separate node blocks for r2, r3, and r4.

```
name: aiops-diamond
topology:
  nodes:
```

```
  h1:
    kind: linux
    image: aiops-multitool:latest
    exec:
      - ip addr add 10.0.1.10/24 dev eth1
      - ip route del default dev eth0
      - ip route add default via 10.0.1.1 dev eth1
  r1:
    kind: linux
    image: frrouting/frr:latest
    binds:
      - r1-frr.conf:/etc/frr/frr.conf
      - daemons:/etc/frr/daemons
  # Separate r2, r3, and r4 node blocks are defined in the full file.
  h2:
    kind: linux
    image: aiops-multitool:latest
    exec:
      - ip addr add 10.0.2.10/24 dev eth1
      - ip route del default dev eth0
      - ip route add default via 10.0.2.1 dev eth1
links:
  - endpoints: ["h1:eth1", "r1:eth1"]
  - endpoints: ["h2:eth1", "r4:eth1"]
  - endpoints: ["r1:eth2", "r2:eth1"]
  - endpoints: ["r1:eth3", "r3:eth1"]
  - endpoints: ["r2:eth2", "r4:eth2"]
  - endpoints: ["r3:eth2", "r4:eth3"]
```

Line or command	What it means and when to use it
name: aiops-diamond	Give the lab its identifier. Generated Docker names use prefix clab-aiops-diamond-.
topology / nodes	Begin the topology and node maps.
h1, h2, r1-r4	Declare two Linux hosts and four Linux routers.
kind: linux	Specify a Linux container node.
image: ...	Choose the container filesystem. Hosts use the locally built image; routers use frrouting/frr.
exec:	List commands Containerlab runs in that node during deployment.
ip addr add .../24 dev eth1	Set the host address and subnet on its lab-facing interface.
ip route del default dev eth0	Remove the base image default route on unused eth0.
ip route add default via ... dev eth1	Send off-subnet host traffic to its attached router.
binds: source:destination	Mount host configuration files into the expected paths inside FRR.
links:	Begin the virtual connection list.
endpoints: [node:interface, node:interface]	Join exactly those two named interfaces into a virtual link.

Topology link	Interface pair
h1 to r1	h1 eth1 <-> r1 eth1
h2 to r4	h2 eth1 <-> r4 eth1
r1 to r2	r1 eth2 <-> r2 eth1
r1 to r3	r1 eth3 <-> r3 eth1
r2 to r4	r2 eth2 <-> r4 eth2
r3 to r4	r3 eth2 <-> r4 eth3

The two middle routes are r1-r2-r4 and r1-r3-r4. OSPF can move traffic to the surviving path when one branch is disabled.

8. FRR routing configuration

The routers use FRRouting (FRR) and OSPF. Interface addresses identify each point-to-point subnet; OSPF advertises those networks so routers learn end-to-end routes.

```
frr defaults traditional
hostname r1
log stdout
interface eth2
 ip address 10.0.12.1/24
!
router ospf
 ospf router-id 1.1.1.1
 network 10.0.12.0/24 area 0
 network 1.1.1.1/32 area 0
!
```

Line or command	What it means and when to use it
frr defaults traditional	Select the FRR command behavior expected by these config files.
hostname r1	Set the router name used in FRR prompts and logs.
log stdout	Send FRR logs to container output.
interface eth2	Enter r1 eth2 interface configuration; this link points to r2.
ip address 10.0.12.1/24	Assign r1 the first address in the r1-r2 subnet.
!	FRR stanza separator.
router ospf	Enter OSPF dynamic-routing configuration.
ospf router-id 1.1.1.1	Assign a stable OSPF identity to r1.
network ... area 0	Enable OSPF on matching interfaces and advertise the subnet in area 0.

9. Deploy and verify

```
sudo containerlab deploy -t diamond.clab.yml
```

Line or command	What it means and when to use it
sudo	Run with system privileges needed to create lab network devices.
containerlab deploy	Create containers and virtual links from the topology.
-t diamond.clab.yml	Select the project topology file.

```
docker ps --format "table {{.Names}}\t{{.Status}}"
docker exec clab-aiops-diamond-h1 ping -c 3 10.0.2.10
docker exec clab-aiops-diamond-h1 iperf3 --version
docker exec clab-aiops-diamond-h2 iperf3 --version
```

Line or command	What it means and when to use it
docker ps	List the containers currently running.
--format ...	Display only the name and status as a readable table.
docker exec NAME COMMAND	Run a program inside an already-running container.
ping -c 3 10.0.2.10	Send three probes from h1 to h2 to validate route and host addressing.
iperf3 --version	Confirm traffic generator is installed on both endpoints.

Do not collect data if the endpoint ping fails. Check container status, FRR logs/config, host addresses and routes first.

10. Linux telemetry commands

```
docker exec clab-aiops-diamond-r2 ip -s link show dev eth2
docker exec clab-aiops-diamond-r2 ip -o link show dev eth2
docker exec clab-aiops-diamond-r2 cat /sys/class/net/eth2/carrier
```

Line or command	What it means and when to use it
docker exec ...	Enter the selected router container to run a Linux command.
ip -s link show dev eth2	Read kernel packet/byte/error/drop counters for r2 eth2; -s adds statistics.
ip -o link show dev eth2	Show one-line interface flags/state; -o makes parsing predictable.
cat .../carrier	Read kernel carrier state: 1 usually means present, 0 absent.

The counter collector uses the kernel ip utility because vtysh did not expose complete counters for the lab's veth links. Sampling is every five seconds.

```
docker exec clab-aiops-diamond-h1 ping -n -i 0.2 -c 5 -W 1 10.0.2.10
```

Line or command	What it means and when to use it
-n	Do not look up hostnames.
-i 0.2	Wait 0.2 seconds between probes.
-c 5	Send five pings per measurement burst.
-W 1	Wait at most one second for each response.
10.0.2.10	h2 destination address.

Because only five pings form each sample, packet-loss estimates are noisy: one packet changes a burst result by 20 percentage points.

```
docker exec clab-aiops-diamond-h2 iperf3 -s -D -p 5202
docker exec clab-aiops-diamond-h1 iperf3 -c 10.0.2.10 -p 5202 -i 1 -t 36000 --forceflush
```

Line or command	What it means and when to use it
-s -D	Run iperf3 as a background server in h2.
-p 5202	Use the lab's dedicated iperf3 port.
-c 10.0.2.10	Connect from h1 to the h2 server.
-i 1	Print one-second throughput intervals.
-t 36000	Keep the client available throughout a long lab run; collector stops it at run end.
--forceflush	Flush interval output so the throughput collector can read it promptly.

11. Linux fault injection commands

tc controls the interface queueing discipline (qdisc). Faults are applied to router egress interfaces, held for about 60 seconds, and removed. The faults.csv schedule records expected labels and time windows; it does not make the live model prediction.

```
docker exec clab-aiops-diamond-r2 ip link set eth2 down
docker exec clab-aiops-diamond-r2 ip link set eth2 up
```

Line or command	What it means and when to use it
ip link set eth2 down	Disable r2 eth2. Routing may fail over to the r3 branch.
ip link set eth2 up	Restore the link after its fault window.

```
docker exec clab-aiops-diamond-r3 tc qdisc add dev eth2 root netem delay 450ms
docker exec clab-aiops-diamond-r3 tc qdisc add dev eth2 root netem loss 15%
docker exec clab-aiops-diamond-r3 tc qdisc del dev eth2 root
```

Line or command	What it means and when to use it
tc qdisc add	Attach a traffic rule to an interface queue.
dev eth2 root	Select eth2 egress and the root queue.
netem delay 450ms	Emulate 450 ms delay.
netem loss 15%	Drop approximately 15 percent of egress packets.
qdisc del ... root	Remove the test rule and restore ordinary queueing.

```
docker exec clab-aiops-diamond-r2 tc qdisc add dev eth2 root tbf rate 1500kbit burst 32000
latency 400ms
```

Line or command	What it means and when to use it
tbf	Token-bucket filter shaper.
rate 1500kbit	Limit sustained egress rate to 1.5 megabits per second.
burst 32000	Allow a short 32,000-byte burst.
latency 400ms	Allow packets to wait up to this amount in the shaper.

The clean repeated profile disables r2-r4 first so traffic traverses r3-r4 for subsequent delay and loss events. If failover bypasses the interface holding a qdisc, the scheduled event may not cause a measured fault. Verify route and telemetry before trusting labels.

12. Collection script and live detector

collect_dataset.sh orchestrates a run. It does not infer the injected label. Its key safeguards and settings are:

```
set -uo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)"
DATA_ROOT="$ROOT/dataset_runs"
RUNS="${1:-1}"
BASELINE_SECONDS="${BASELINE_SECONDS:-60}"
LIVE_DETECTOR="${LIVE_DETECTOR:-0}"
PYTHON="${PYTHON:-python3}"
```

Line or command	What it means and when to use it
set -uo pipefail	Catch use of unset variables and failures in command pipelines; this script checks many commands explicitly.
ROOT=...BASH_SOURCE...pwd	Find the script directory so relative project paths are reliable.
DATA_ROOT=...dataset_runs	Store captures below the project.
${1:-1}	Use argument 1, or default to one run.
${BASELINE_SECONDS:-60}	Use an environment override or default to 60 seconds.
${LIVE_DETECTOR:-0}	Do not start live scoring unless enabled with 1.
${PYTHON:-python3}	Use a selected interpreter or system python3.

```
for node in h1 h2 r1 r2 r3 r4; do
  docker inspect -f ... "clab-aiops-diamond-$node"
done

trap cleanup_lab EXIT
```

Line or command	What it means and when to use it
for node ...; do	Loop through all six expected lab containers.
docker inspect	Check the engine says each container is running before collection.
done	Finish the node-check loop.
trap cleanup_lab EXIT	Call cleanup on every normal or error exit. Cleanup stops collector/iperf processes, brings the configured r1 link up, and removes test qdiscs.

For every run, the script creates a UTC-stamped unique directory, writes run metadata, generates collector copies targeting that directory, starts iperf3, and launches four timestamped sensors. With LIVE_DETECTOR=1 it loads model.json and emits predictions at five-second intervals. After the baseline it executes the fault schedule, collects a post-fault baseline, stops background work, and checks that telemetry.csv, link_state.csv, rtt.csv, throughput.csv, and faults.csv exist and are nonempty.

```
cd ~/aiops-project
LIVE_DETECTOR=1 PYTHON=./.venv/bin/python bash collect_dataset.sh 1
```

Line or command	What it means and when to use it
cd ...	Run in the project folder.
LIVE_DETECTOR=1	Enable streaming predictions during the capture.
PYTHON=...	Use the pinned project environment.
bash collect_dataset.sh	Start the run orchestrator.
1	Request one capture; each capture has its own folder.

The detector predicts from telemetry features such as counter rates, RTT, measured ping loss, link state, throughput, and interface identity. The separate offline evaluator joins timestamps to faults.csv to score labels after collection. The detector does not read that schedule. An observed interface state DOWN is a direct measured signal and triggers the link_down rule.

13. Troubleshooting and safe shutdown

Problem	What it means and next check
docker ps empty	Docker works but nothing is running yet. Deploy the topology.
Docker socket error	Start Docker Desktop; turn on Ubuntu WSL Integration; reopen Ubuntu; inspect docker context ls and docker version.
h1 cannot ping h2	Do not capture. Check all containers, host IP/default route commands, FRR configuration and OSPF.
iperf3 missing	Rebuild aiops-multitool image, then redeploy the topology.
qdisc preflight not clean	Inspect tc qdisc show on r2/r3 eth2. Remove only stale test qdiscs before a run.
Unexpected class/score	Check if route failover bypassed the injected interface; validate measured signal and event label.

```
docker exec clab-aiops-diamond-r2 tc qdisc show dev eth2
docker exec clab-aiops-diamond-r1 ip -br link
sudo containerlab destroy -t diamond.clab.yml
```

Line or command	What it means and when to use it
tc qdisc show	Inspect shaping rules before/after tests.
ip -br link	Show compact interface states.
containerlab destroy	Stop and remove this lab's containers and links.
-t ...	Select the topology to remove; dataset_runs files are separate and remain on disk.

Let collect_dataset.sh finish or interrupt it with Ctrl+C so its cleanup trap can run. Inspect links and qdiscs after interrupted tests. Destroy the topology when done; do not delete dataset_runs if you need the recorded measurements.

14. Official setup references

Area	Official documentation
Docker Desktop WSL engine and integration	https://docs.docker.com/desktop/features/wsl/
Docker development from WSL	https://docs.docker.com/desktop/features/wsl/use-wsl/
Microsoft WSL install	https://learn.microsoft.com/en-us/windows/wsl/install

Area	Official documentation
Microsoft WSL commands	https://learn.microsoft.com/en-us/windows/wsl/basic-commands
Containerlab installation	https://containerlab.dev/install/
Python venv	https://docs.python.org/3/library/venv.html

Installation UI and installer behavior may change. Use these official references for current platform requirements. The command explanations and project paths above describe the saved AIOps lab as of 13 October 2026.